Tutorato 2 - Esercizi Extra

Matteo Mazzaretto

27 Ottobre 2026



Extra 1: crocette

1. Una variabile `static C ciao;` dichiarata a livello di file ha:
A) linkage esterno: è visibile in tutti i file
B) linkage interno: il nome è visibile solo in questo file
C) durata limitata al blocco in cui è dichiarata

2. Il venditore consegna `orario.h` e `orario.o`. Chi usa `orario.h` e a cosa serve `orario.o`?
A) l'header serve al linker, il `.o` al compilatore
B) l'header serve al compilatore, il `.o` al linker
C) servono entrambi solo al compilatore

3. Nell'header si scrive `class orario_rappr;` seguita da `orario_rappr r;`. Cosa succede?
A) errore: il tipo è incompleto e il compilatore non ne conosce la dimensione
B) compila, ma `orario_rappr` resta invisibile
C) errore solo al momento del linking

Extra 1: soluzione

1 → B. `static` a livello di file cambia solo la visibilità (linkage interno). La durata è la stessa di una globale: fino alla fine del programma.

2 → B. In compilazione al compilatore basta l'header (sapere che classi e metodi esistono). Il linker collega le chiamate al codice che si trova in `orario.o`.

3 → A. Per creare un oggetto serve conoscerne la dimensione, quindi la definizione completa. Con una dichiarazione incompleta si può usare solo un **puntatore** (o un riferimento): per questo l'information hiding usa `orario_rappr*`. L'errore è del compilatore, non del linker.

Extra 2: cosa stampa?

```
class T{
  public:
    int id;
    T(int x = 7): id(x){ cout << "C" << id << " "; }
    ~T(){ cout << "D" << id << " "; }
};

int main(){
  T a[2];
  T* p = new T[3];
  for(int j = 0; j < 3; j++) p[j].id = j;
  delete[] p;
  cout << "fine ";
}
```

Extra 2: soluzione

Output:

```
C7 C7 C7 C7 C7 D2 D1 D0
fine D7 D7
```

- Il costruttore di default (`x = 7`) viene chiamato per ogni elemento: 2 volte per `a`, 3 per `p`
- Poi gli `id` di `p` diventano 0, 1, 2
- `delete[] p` distrugge gli elementi in ordine **inverso**: `D2 D1 D0`
- L'array statico `a` viene distrutto alla fine del `main`, dopo `fine`: `D7 D7`

Extra 3: crocette

1. Quale cast controlla la validità della conversione a **run-time**?

A) `static_cast`

B) `reinterpret_cast`

C) `dynamic_cast`

2. `const int x = 3;` poi `const_cast<int&>(x) = 5;`. Cosa succede?

A) errore di compilazione

B) compila e il comportamento è indefinito, perché `x` è dichiarata `const`

C) compila e `x` vale sicuramente 5

3. Un cast stile C, ad esempio `(int*) p`, cosa può fare?

A) solo un `static_cast`

B) prova `static_cast`, `const_cast` e `reinterpret_cast` in sequenza, anche nascondendo conversioni pericolose

C) solo un `dynamic_cast`

Extra 3: soluzione

1 → C. `static_cast` e `reinterpret_cast` si basano sul tipo statico (compile-time). Solo `dynamic_cast` controlla a run-time e restituisce `nullptr` se non è valido (per i puntatori).

2 → B. Il cast compila, ma modificare un oggetto **dichiarato** `const` è comportamento indefinito. Il caso lecito è quando l'oggetto originale non è `const` e lo si raggiunge tramite un riferimento o puntatore `const`.

3 → B. Per questo i cast stile C sono sconsigliati: non si vede quale conversione viene davvero applicata.

Extra 4: cosa stampa?

```
class C{
  private: string nome;
  public:
    C(const string& n): nome(n){ cout << "+" << nome << " "; }
    ~C(){ cout << "-" << nome << " "; }
};

C g("g");

void f(int k){
  static C s("s");
  C l("l");
  cout << k << " ";
}

int main(){
  cout << "start ";
  f(1);
  f(2);
  static C m("m");
  cout << "end ";
}
```

Extra 4: soluzione

Output:

```
+g start +s +l 1 -l +l 2
-l +m end -m -s -g
```

- `g` è globale: costruita prima di `main`
- `s` è static locale: costruita alla **prima** chiamata di `f` e mai più
- `l` è locale: costruita e distrutta a **ogni** chiamata (`+l 1 -l`, `+l 2 -l`)
- `m` è static locale di `main`: costruita quando l'esecuzione arriva a quella riga
- Alla fine le durate statiche si distruggono in ordine inverso di costruzione: `m`, `s`, `g`

Extra 5: cosa stampa?

```
int main(){
  double d = 7.9;
  int i = static_cast<int>(d);
  cout << i << " ";

  int y = 3;
  const int& r = y;
  const_cast<int&>(r) = 10;
  cout << y << " " << r << " ";

  cout << static_cast<double>(7 / 2) << " ";
  cout << static_cast<double>(7) / 2;
}
```

Extra 5: soluzione

Output:

```
7 10 10 3 3.5
```

- `static_cast<int>(7.9)` tronca: `i = 7`
- `r` è un riferimento a `y` (che non è `const`): `const_cast<int&>(r) = 10` modifica `y`, quindi `y` e `r` valgono 10
- `7 / 2` è una divisione tra interi (3) e il cast viene dopo: `3`
- `static_cast<double>(7) / 2` divide in virgola mobile: `3.5`

Extra 6: completa

UNIVERSITÀ DI PADOVA

```
class Playlist{
  friend /* 1 */ operator<<(/* 2 */ os, /* 3 */ p);
  private:
    class Brano{
      public:
        int dur;
        Brano* next;
        Brano(int d, Brano* n): dur(d), next(n) {}
    };
    Brano* first;
  public:
    Playlist(): first(nullptr) {}
    void add(int d){ first = new Brano(d, first); }
};

/* 1 */ operator<<(/* 2 */ os, /* 3 */ p){
  Playlist::Brano* q = /* 4 */;
  while(q){
    os << q->dur << " ";
    q = /* 5 */;
  }
  return os;
}

int main(){
  Playlist p;
  p.add(3); p.add(5); p.add(8);
  cout << p;
}
```

Extra 6: soluzione

```
/* 1 */ ostream&
/* 2 */ ostream&
/* 3 */ const Playlist&
/* 4 */ p.first
/* 5 */ q->next
```

Output:

```
8 5 3
```

- Il tipo di ritorno e il primo parametro sono `ostream&`, per poter concatenare `cout << a << b`
- `const Playlist&` evita la copia e garantisce di non modificare la playlist
- Grazie a `friend` la funzione legge `p.first` e usa anche il tipo privato `Playlist::Brano`
- L'inserimento è in testa, quindi si stampa 8 5 3

Extra 7: crocette

1. Perché `end()` restituisce un iteratore con `punt == nullptr`?
A) perché punta all'ultimo nodo della lista
B) perché indica la posizione "oltre l'ultimo", che non è un elemento valido
C) perché è l'unico valore che `punt` può avere nella classe

2. Si scrive `C* p = new C[3];` e poi `delete p;`. Cosa succede?
A) comportamento indefinito: tipicamente non vengono chiamati tutti i distruttori
B) viene chiamato il distruttore su tutti e 3 gli elementi
C) errore di compilazione

3. Una funzione dichiarata `friend` di una classe:
A) è un metodo della classe, con il puntatore `this`
B) non è un metodo, ma può accedere ai membri privati
C) può accedere ai membri privati solo degli oggetti `const`

Extra 7: soluzione

1 → B. L'iteratore `end()` non punta a un `Reel` valido: serve come segnale di fine, e `++` sull'ultimo elemento ci arriva portando `punt` a `nullptr`. Usare `x[end()]` è un errore.

2 → A. Un array creato con `new[]` si libera con `delete[]`. Con `delete` semplice il comportamento è indefinito (tipicamente si distrugge solo un elemento).

3 → B. Una funzione `friend` è una funzione esterna: non ha `this`, deve ricevere l'oggetto come parametro, ma può leggere e scrivere i membri privati, `const` o no.

Extra 8: cosa stampa?

Usa la classe `Instagram` con iteratore vista nelle slide. `aggiungi_in_testa` inserisce all'inizio della lista. Un `bool` stampato con `cout` vale `0` o `1`.

```
int main(){
  Instagram x;
  x.aggiungi_in_testa(10);
  x.aggiungi_in_testa(20);
  x.aggiungi_in_testa(30);

  Instagram::iteratore it = x.begin();
  cout << x[it++] << " ";
  cout << x[it] << " ";
  cout << x[++it] << " ";
  ++it;
  cout << (it == x.end()) << " ";
  cout << (++it == x.end());
}
```

Extra 8: soluzione

Output:

```
30 20 10 1 1
```

Lista: 30 ⟶ 20 ⟶ 10

- `x[it++]`: il postfisso restituisce una **copia** con il valore precedente, quindi stampa 30 e poi `it` avanza su 20
- `x[it]`: stampa 20
- `x[++it]`: il prefisso avanza **prima**, quindi stampa 10
- Il `++it` successivo porta `it` a `nullptr`: `it == x.end()` vale 1
- `++` su un iteratore già a `nullptr` non fa nulla (c'è `if (punt)`): ancora 1

Extra 9: completa

UNIVERSITÀ DI PADOVA

Completa gli spazi `/* 1 */` ... `/* 6 */`.

```
// dentro la classe iteratore
iteratore& operator++(){
  if(/* 1 */) punt = punt->next;
  return /* 2 */;
}

iteratore operator++(int){
  /* 3 */ aux = *this;
  ++(*this);
  return /* 4 */;
}

// fuori dalla classe: quanti Reel durano piu' di k secondi?
int Conta_Lunghi(const Instagram& x, int k){
  int n = 0;
  for(Instagram::iteratore it = /* 5 */; it != /* 6 */; ++it)
    if(x[it] > k) n++;
  return n;
}
```

Extra 9: soluzione

```
/* 1 */ punt
/* 2 */ *this
/* 3 */ iteratore
/* 4 */ aux
/* 5 */ x.begin()
/* 6 */ x.end()
```

- Prefisso: avanza e ritorna l'iteratore stesso, per **riferimento** (`*this`)
- Postfisso: salva una copia (`aux`), avanza riusando il prefisso e ritorna la copia **per valore**
- Il controllo `if(punt)` evita di leggere `nullptr->next`
- `x` è `const`: `begin()` e `end()` sono metodi `const`, quindi si possono chiamare
- Per esempio con durate 70, 50, 10 e `k = 40` il risultato è 2

Extra 10: cosa stampa?

```
class C{
  public:
    char id;
    C(char i = 'x'): id(i){ cout << "+" << id << " "; }
    ~C(){ cout << "-" << id << " "; }
};

C g('G');

C* make(){
  C* p = new C[2];
  p[0].id = 'P';
  p[1].id = 'Q';
  return p;
}

int main(){
  C a[2] = { C('A'), C('B') };
  C* p = make();
  {
    C b('b');
    delete[] p;
  }
  cout << "fine ";
}
```

Extra 10: soluzione

Output:

```
+G +A +B +x +x +b -Q -P
-b fine -B -A -G
```

- `G` è globale: costruita prima di `main`
- `a` costruisce `A` e `B`; `make()` costruisce due `x` con il costruttore di default (poi cambiati in `P` e `Q`)
- `b` viene costruita nel blocco: `+b`
- `delete[] p` distrugge in ordine inverso: `-Q -P`, poi esce dal blocco: `-b`
- `fine`, poi alla fine del `main` l'array `a` si distrugge al contrario (`-B -A`) e infine la globale (`-G`)